Assignment 2: Supply-Chain Malware Detection

- Assigned: 20/11/25
- Due: 29/11/25

Introduction

In this assignment you are your team is going to simulate a supply-chain attack in Python and detect it using OSSEC-HIDS and YARA. The purpose of this assignment is to give you hands-on experience with how attackers can hide malicious behavior in trusted libraries and how defenders can detect such activity using host-based intrusion detection and malware classification tools.

Basic Theory

A supply-chain attack targets the less secure parts of a trusted software supply chain, such as third-party libraries. Instead of attacking a system directly, the attacker compromises a dependency (for example, a Python package) that developers use in their software.

Fabric is a Python package used for remote command execution, commonly for deployment or automation. Paramiko is the SSH communication layer used by Fabric. According to Fabric's documentation:

Fabric is a high level Python (2.7, 3.4+) library designed to execute shell commands remotely over SSH, yielding useful Python objects in return. It builds on top of Invoke (subprocess command execution and command-line features) and Paramiko (SSH protocol implementation), extending their APIs to complement one another and provide additional functionality.

Source code of Fabric: https://github.com/fabric/fabric

In a supply-chain attack, a malicious version of Paramiko could replace the original one, and Fabric would unknowingly execute it. In real defensive scenarios, such malicious modifications are often detected using:

1. Host-based intrusion detection systems such as OSSEC-HIDS, which monitor file changes and suspicious system activity.
2. Malware-analysis tools like YARA to identify tampered or malicious files by matching patterns or signatures inside the modified code.

The Scenario

In this assignment you will first take the role of the "bad" guy that somehow managed to get (steal..) the github credentials of a user maintaining paramiko. You will alter paramiko and add a small piece of code that does something "bad". The first part of this assignment is for you to locate the right part in the paramiko source code and add this "bad" part.

Of course, the maintainers of fabric are unaware of this alteration and they continue to use paramiko as their dependency. As an effect when fabric uses paramiko, they use the "bad" version and "bad" things happen. You will have to locate in which part of fabric's source code, the paramiko code is used.

After alrering paramiko and making sure that it does bad things. You alter role and you become the "good" guy. You will have to use the OSSEC-HIDS (Host-based intrusion detection system) and the yara tool to detect the malware.

STEP 1 - Set Up the Environment

Install Python and tools:

```
sudo apt update
sudo apt install python3 python3-pip python3-venv git
```

Create a virtual environment AND activate it (IMPORTANT STEP):

```
python3 -m venv venv
source venv/bin/activate
```

Install the official Fabric package:

```
pip install fabric
```

STEP 2 - Setup a local SSH environment (for testing)

You need to setup an SSH environment. This will be used to test your implementation. For this project this can be you local computer. For this reason you will need to be able to SSH to your terminal. Some instruction are given here. Feel free to alter these instruction to meet your computer's configuration:

- Mac
  - Open System Settings
  - Go to General -> Sharing
  - Enable Remote Login
  - This immediately starts the built-in SSH server (sshd) and allows you to connect to your Mac via:

    ```
    ssh yourusername@localhost
    ```

  - If this connection fails check that the SSH service is running via:

    ```
    sudo systemsetup -getremotelogin
    ```

- Linux
  - Instal openssh:
  - `sudo apt install openssh-server`
  - `sudo systemctl enable ssh`
    `sudo systemctl start ssh`

- Then test:

```
ssh localhost
```

- Windows
  - Install the OpenSSH Server feature:
    - Settings -> Apps -> Optional Features -> View features
    - Search for OpenSSH Server -> Install.
  - Alternatively you can install it from PowerShell:
  - `Get-WindowsCapability -Online | Where-Object Name -like 'OpenSSH.Server*'`
  - `Add-WindowsCapability -Online -Name OpenSSH.Server~~~~0.0.1.0`
  - Start and enable the SSH service:
    - In PowerShell (Run as Administrator):
    - `Start-Service sshd`
      `Set-Service -Name sshd -StartupType 'Automatic'`
    - Optionally start the authentication agent:
    - `Start-Service ssh-agent`
      `Set-Service -Name ssh-agent -StartupType 'Automatic'`
  - Allow SSH through the firewall:

```
New-NetFirewallRule -Name sshd -DisplayName 'OpenSSH Server' -
Enabled True -Direction Inbound -Protocol TCP -Action Allow -
LocalPort 22
```

  - Test from the same machine:

```
ssh your_windows_username@localhost
```

  - Notes:
    - The Windows SSH server uses C:\ProgramData\ssh\sshd_config for configuration.
    - The first time you connect via SSH, Windows may ask to trust the host key.
    - Your Windows user password is used for login (unless you enable key-based auth).
- Docker
  - If you prefer not to touch your system services, run a temporary SSH server using Docker:

```
docker run -d -p 2222:22 --name test-ssh rastasheep/ubuntu-sshd
```

  - Then SSH into it:
  - `ssh root@localhost -p 2222`
    `# password: root`

STEP 3 - Install to the local SSH environment through fabric

In the python environment that you created in STEP 1, run the following:

```
from fabric import Connection

c = Connection("admin@localhost") # Change the "admin" to your username
c.run("echo Running on local machine")
```

You should be able to see:

```
Running on local machine
<Result cmd='echo Running on local machine' exited=0>
```

If you reached this stage, the setup is complete.

STEP 4 - Locate where fabric Paramiko is installed

The main task of this assignment is to alter (tamper) the source code of paramiko so that when the fabric library is using it then, it should do something "bad".

This is how you can see where the local installation of paramiko is:

```
import os
import fabric
import paramiko

print(os.path.dirname(fabric.__file__))
print(os.path.dirname(paramiko.__file__))
```

You should investigate fabric's source code and figure out how fabric is using paramiko. Then you should change the part of paramiko that is used by fabric when the command `c.run(...)` is executed.

After tampering paramiko, when the user runs a command in a remote computer through fabric, then an arbitrary "bad" command should also be run. For example:

```
from fabric import Connection

c = Connection("admin@localhost") # Change the "admin" to your username
c.run("echo Running on local machine")
```

When the user executes this script, apart of the command `echo Running on local machine`, the "bad" command should also run. You can use the following "bad" command:

```
echo "You are PWNED!"
```

Some more information about PWNED

Notice that this command should run in the remote computer that the user is trying to run his/her command.

So although the user did not explicitly installed paramiko (notice that we only did `pip install fabric`) a compromised version of paramiko was installed.

STEP 5. Easy version

Up to that point the "bad" thing that paramiko does is to print a message in the remote computer. This is harmless so let's make things serious. You should change paramiko so that after printing the message (you are PWNED) paramiko should search in the remote computer for an installed version of paramiko. If it finds an installed version then it "infects" it (changes it) so that when it is used then it does the same as this changed version of paramiko. In other words you should change paramiko so that it looks for other versions of paramiko in remote computers and change them too.

STEP 5 . Difficult version (optional)

STEP 5 as it is described above does not have a viral behavior. Once your altered version of paramiko runs, it alters other versions of paramiko that are installed in the remote computer that the user wants to access. But this infection stops there. If these (in the remote computer) versions of paramiko are run, then they will not infect other versions of paramiko. An interesting task is to alter paramiko so that when it runs it infects other versions of paramiko as well!

At this point you might be wondering how you can change a program by adding them to run code X, and when X runs, it changes other programs by adding them the code X ?

Writing programs to print themselves are called quines. Here is a quine in python:

```
msg1 = "A message"
msg2 = f"Another message that includes msg1={msg1} and a backslash: \\"
q = 'msg1 = "A message"\nmsg2 = f"Another message that includes msg1={msg1}
and a backslash: \\\\\"\nq = %r\nprint(q %% q)'
print(q % q)
```

So you will have to write a quine that prints "you are PWNED" and infects other installed version of paramiko to print "you are PWNED" and infect other installed versions of paramiko... and so on!

Note: This is a particularly defficult task!

STEP 6 . Install OSSEC-HIDS

Install and Configure OSSEC-HIDS. This tool monitors file changes and suspicious process activity on a remote system (OSSEC-HIDS). To install on ubuntu:

```
sudo apt install ossec-hids
```

You also install it from source from github

Monitor critical files: Add your project directory (e.g., `~/HY413_2025/`) to OSSEC's local_files list in `/var/ossec/etc/ossec.conf` to watch for modifications and then restart OSSEC:

```
sudo systemctl restart ossec
```

Observe alerts in: `/var/ossec/logs/alerts/alerts.log` . OSSEC should flag suspicious modifications made by your tampered Paramiko during tests.

Note: OSSEC-HIDS cannot be fully built on macOS. In my system I was able to run it by installing an older version: 2.8.3

If you cannot install OSSEC-HIDS you can use a simple file system monitoring tool. Options are:

- fswatch
- inotifywatch. Only for Linux
- watchdog. Pure python implementation
- watchman. facebook's implementation of a fs monitoring system

STEP 7 . Install YARA.

Install YARA

YARA is a tool aimed at (but not limited to) helping malware researchers to identify and classify malware samples.

You can download the source code from here and do the usual steps:

```
tar zxvf yara-4.5.5.tar.gz
cd yara-4.5.5
./build.sh  # You can open the file and run the contained steps
indivudually
```

STEP 8 . Create a yara file

YARA, except from a malware detection tool is also a format for describing malwares. Online there are plenty of repositories containing yara files for a big range of malware.

In this step you will create a yara file for the malware that you created. Here is a documentation page on creating yara files, although it is fairly easy. An example of yara file looking for files that contain the string "I am bad" is:

```
rule test_rule : banker
{
    meta:
        description = "This is just an example"
        threat_level = 3
        in_the_wild = true
    strings:
        $a = "I am bad"
        $b = "i'm bad"
    condition:
        $a or $b
}
```

You can test a yara file:

- Save the above text in a file. i.e. `test_1.yara`
- Create a file in a directory `my_precious_dir/lyrics.txt` with the following contents:
- `Because I'm bad, I'm bad, shamone (bad, bad, really, really bad)`
- `You know I'm bad, I'm bad (bad, bad)`
- `You know it (really, really bad)`
- `You know I'm bad, I'm bad (bad, bad)`
- `Shamone, you know (really, really bad)`
- `And the whole world has to answer right now`
- `Just to tell you once again`
- `Who's bad?`
- Run:

```
./yara -r test_1.yara my_precious_dir
```

- You should see:

```
test_rule my_precious_dir/lyrics.txt
```

In this part you should write a YARA file that detects your malware. Alsop demonstrate that the yara tool can detect it.

STEP 9

Write the Report: Include detection evidence and mitigation strategies in a 2-3 page technical report that includes:

- Overview: Briefly describe the simulated supply-chain attack scenario.
- Implementation: Summarize how you modified Paramiko to behave maliciously.
- Detection Evidence: Include logs or screenshots from:
  - OSSEC-HIDS
  - YARA output (CLI or Python API)
- Mitigation Strategies: Propose how similar attacks could be prevented in real systems (e.g., hash verification, runtime monitorin)